上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



Chapter 4

Unit Test environment and tools

1. Java
2. JUNIT
3. EMMA
4. IDEA
5. Mock
6. 智能化测试方法——以大模型为例

Unit Test environment: Java

Why is Java ?

- Easy to get source code and documentation
- Completed test environment
- More free test tools

安装JDK（ Java Development Kit ）

- JDK 是整个Java的核心，包括了Java运行环境，Java工具和Java基础的类库。
- 没有JDK的话，无法安装或者运行java程序
- JDK共有3个版本，分别为：
  - SE(Jave SE)，standard edition，标准版
  - EE(Jave EE)，enterprise edition，企业版
  - ME(Jave ME)，micro edition，主要用于移动设备、嵌入式设备上的java应用程序
- 推荐下载JDK版本：Java SE Development Kit

安装与配置

- 下载与安装，假设安装到C:\Program Files\Java\jdk1.8.0_60
- 配置PATH：相当于C++中的GCC配置
  - 我的电脑点右键，选择“属性”，选择“高级”标签，进入“环境变量设置，在系统变量里找到PATH变量，选择→编辑 ，里面已经有很多的变量值, 变量值的最前面加上C:\Program Files\Java\jdk1.8.0_60\bin;
- 配置CLASSPATH：寻找标准类库位置
  - 我的电脑点右键，选择“属性”，选择“高级”标签，进入环境变量设置，在系统变量那一栏中点新建→ CLASSPATH ，并且设置
  - classpath=.; C:\Program Files\Java\jdk1.8.0_60\lib;C:\Program Files\Java\jdk1.8.0_60\lib\tools.jar;

```
→  ~ java -version
java version "17.0.1" 2021-10-19 LTS
Java(TM) SE Runtime Environment (build 17.0.1+12-LTS-39)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.1+12-LTS-39, mixed mode, sharing)
```

1. Java
2. JUNIT
3. EMMA
4. IDEA
5. Mock
6. 智能化测试工具

JUNIT

- JUnit是由 Erich Gamma 和 Kent Beck 编写的一个回归测试框架（regression testing framework）。
- Junit测试是程序员开发测试工具，即所谓白盒测试，因为程序员知道被测试的软件如何（How）完成功能和完成什么样（What）的功能。Junit是一套框架，继承TestCase类，就可以用Junit进行自动测试了。
- http://www.junit.org

JUNIT

- JUnit是由 Erich Gamma 和 Kent Beck 编写的一个回归测试框架（regression testing framework）。
- Junit测试是程序员开发测试工具，即所谓白盒测试，因为程序员知道被测试的软件如何（How）完成功能和完成什么样（What）的功能。Junit是一套框架，继承TestCase类，就可以用Junit进行自动测试了。
- http://www.junit.org

JUNIT安装

- 1. 从http://www.junit.org/ 下载junit.jar
- 2. 将JAR文件拷贝到JDK的LIB目录， C:\Java\jdk1.8.0_60\lib.。
- 3. 在CLASSPATH 目录下增加一条 c:\Java\jdk1.8.0_60\lib\junit.jar;

- 如采用vs code，还需要修改settings.json[1]
- eclipse中已经集成了JUnit 3 和JUnit 4。
- IDEA中已集成了Junit4和Junit 5
- Junit 5需要JDK 8及以上的版本

常见Junit5标注

注解	描述
@Test	表示方法是一种测试方法。
@ParameterizedTest	表示方法是参数化测试
@RepeatedTest	表示方法是重复测试模板
@DisplayName	为测试类或者测试方法自定义一个名称
@BeforeEach	表示方法在每个测试方法运行前都会运行
@AfterEach	表示方法在每个测试方法运行之后都会运行
@BeforeAll	表示方法在所有测试方法之前运行
@AfterAll	表示方法在所有测试方法之后运行
@Disabled	用于禁用测试类或测试方法

常见Junit测试代码结构

```
public class CalculatorTest {
    private Calculator calculator;

    @BeforeEach
    void setUp()
    {
        calculator=new Calculator();
    }

    @AfterEach
    void tearDown()
    {

    }

    @Test
    void testAddMethod()
    {
        assertEquals( expected: 2,calculator.add( a: 1, b: 1));
    }
}
```

JUnit会以以下顺序执行测试：（大致的代码）

```
try {
        HelloTest test = new HelloTest(); // 建立测试类实例
        test.setUp(); // 初始化测试环境
        test.testAdd(); // 测试某个方法
        test.tearDown(); // 清理资源
}
catch…
```

- setUp()是建立测试环境，这里创建一个Hello类的实例；tearDown()用于清理资源，如释放打开的文件等等。以test开头的方法被认为是测试方法，JUnit会依次执行testXxx()方法。
- 在testAbs()方法中，我们对abs()的测试分别选择正数，负数和0，如果方法返回值与期待结果相同，则assertEquals不会产生异常。
- 如果有多个testXxx方法，JUnit会创建多个XxxTest实例，每次运行一个testXxx方法，setUp()和tearDown()会在testXxx前后被调用，因此，不要在一个testA()中依赖testB()。

- JUnit通过单元测试，能在开发阶段就找出许多Bug，并且，多个Test Case可以组合成Test Suite，让整个测试自动完成，尤其适合于XP方法。

- 每增加一个小的新功能或者对代码进行了小的修改，就立刻运行一遍Test Suite，确保新增和修改的代码不会破坏原有的功能，大大增强软件的可维护性，避免代码逐渐“腐烂”。

1. Java
2. JUNIT
3. EMMA
4. IDEA
5. Mock
6. 智能化测试工具

EMMA

- 测试覆盖率（Code Coverage）
- 测试覆盖率，简单的说，就是评价测试活动覆盖产品代码的指标。测试的目的，是确认产品代码按照预期一样工作，也可以看作是产品代码工作方式的说明文档。进一步考虑，测试覆盖率可以看作是产品代码质量的间接指标——之所以说是间接指标，因为测试覆盖率评价的是测试代码的质量，并不是产品代码的质量。
- 代码覆盖率是一种**白盒测试**，因为测试覆盖率是评价产品代码类内部的指标，而不是评价系统接口或规约。测试覆盖率尤其用于评价测试代码是否已经覆盖了产品代码所有的路径。

EMMA

- EMMA 是一种用来衡量java软件覆盖率的软件。它对于不可达代码的检测，以及验证测试集和交互式测试对程序的覆盖情况是很必要的。
- EMMA的设计目标有以下几个，
- 1. 在不增加创建或执行开销的情况下，提供**丰富的覆盖分析数据**；
- 2. 促进团队开发效率，同时，也能加快单元开发－测试周期；
- 3. 促进小型java软件的开发效率，同时，提高包含大量类的大型企业软件的开发。
- EMMA与其他覆盖测试工具的区别在于它面向极限编程，目的是促使循环的开发－测试软件开发模型。

- 衡量测试覆盖率的指标很多，常用的指标有：
- Statement coverage，也称作Line coverage，用于评价测试的**代码语句覆盖率**。
- Basic block coverage，是Statement coverage的一个变种，它把**没有一个分支的代码区域**作为一个计量单位，而不是简单的代码行，用于一个if-else分支代码行数远远大于另一个的情况，在这种情况下，statement coverage指标并不适用。
- Decision coverage（也称作Branch coverage），用于评价代码分支地测试覆盖率。
- Path coverage，和Decision coverage相似，用于评价**代码从开始到结束所有路径**的测试覆盖率。
- Function coverage，用于评价代码方法的测试覆盖率。
- EMMA目前支持四种Coverage类型：
- class、method、line和basic block。

1. Java
2. JUNIT
3. EMMA
4. IDEA
5. Mock
6. 智能化测试工具

IntelliJ IDEA

- Every aspect of IntelliJ IDEA is specifically designed to maximize developer productivity.
- Together, powerful static code analysis and ergonomic design make development not only productive but also an enjoyable experience.

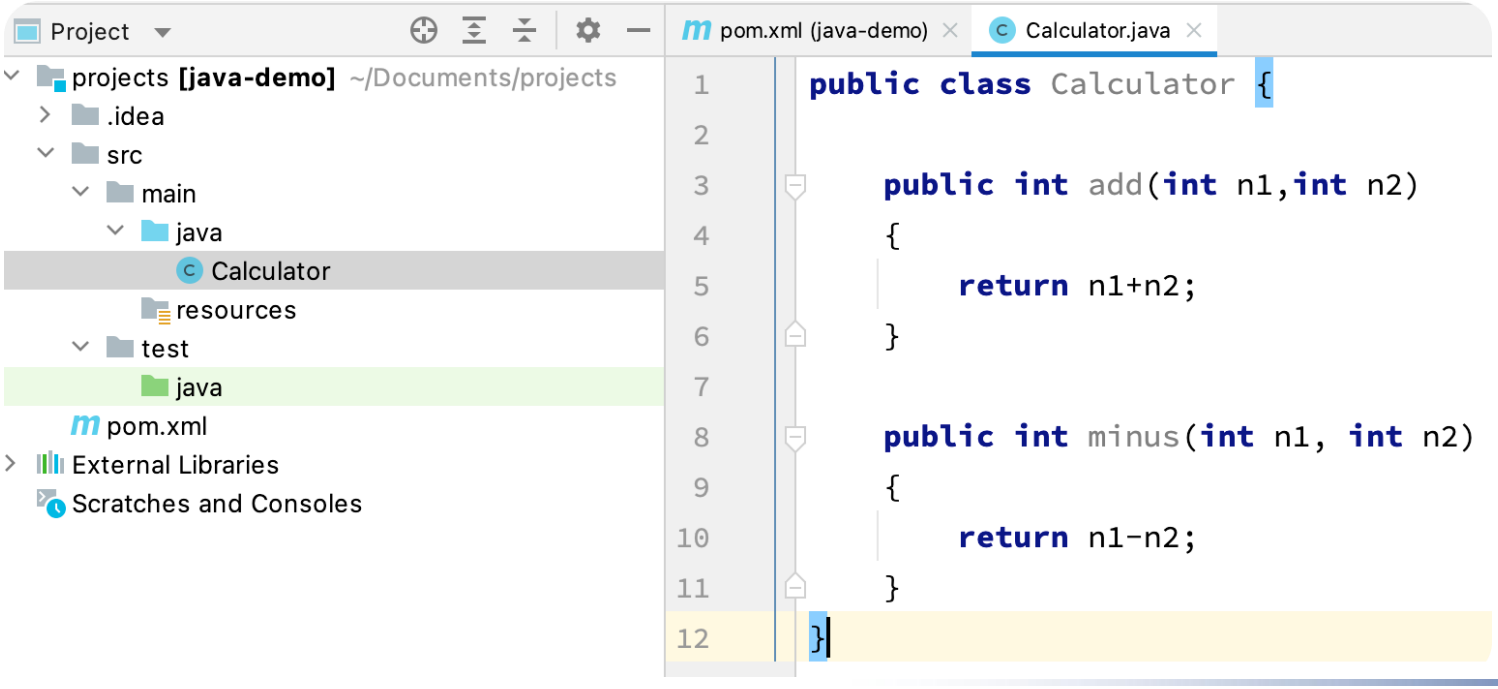


Junit on IntelliJ

- 内置Junit4&5
- 内置测试覆盖度工具
- 测试类生成插件
- 导出测试报告及测试覆盖率报告

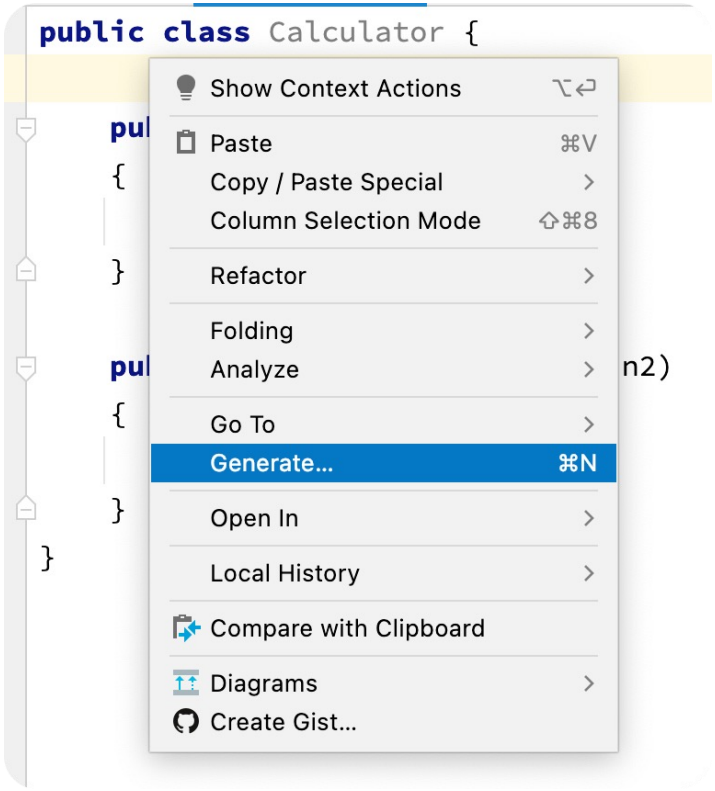
Junit on IntelliJ

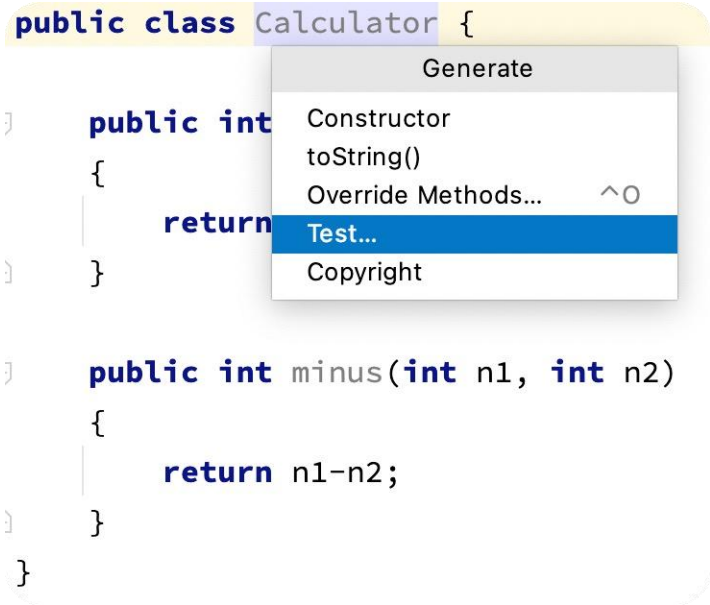
- 新建java-demo项目，采用maven进行包管理
- 待测对象：Calculator类
- 为了保持良好的代码结构，一般将测试类与主程序放置在不同的根目录下



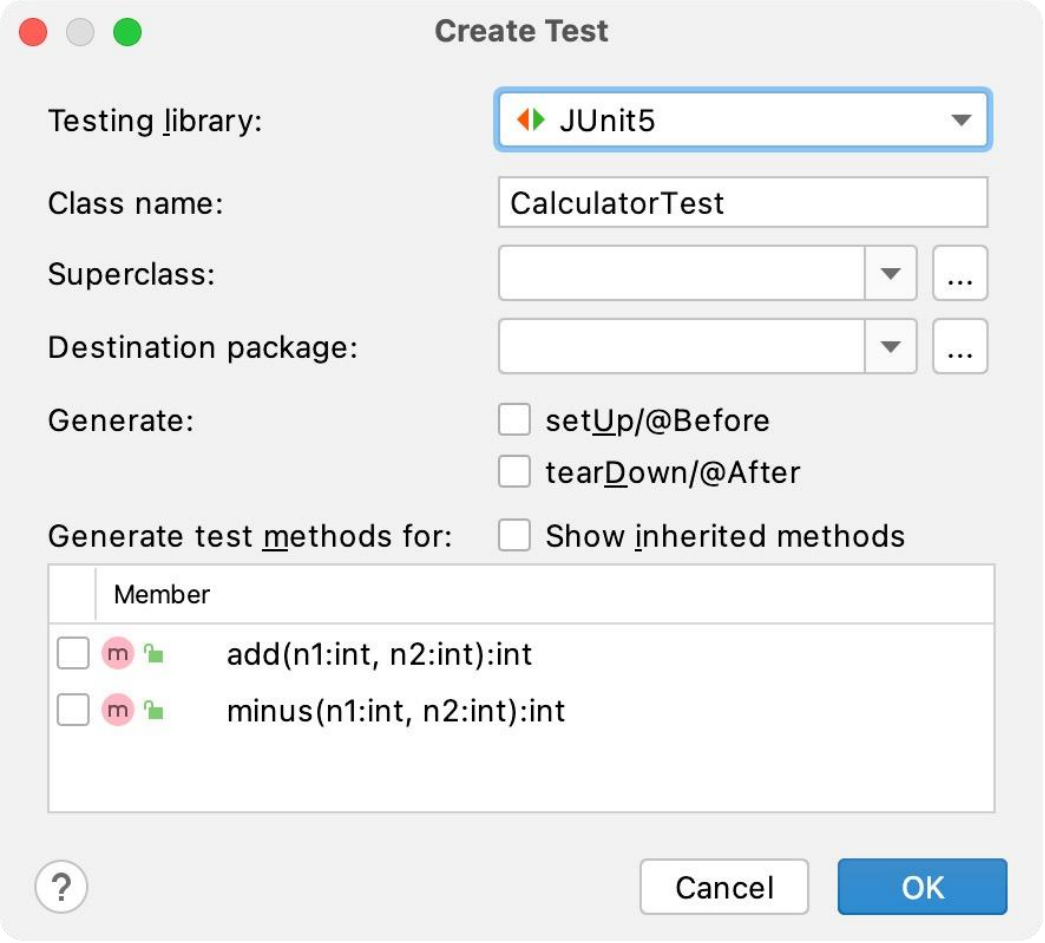
创建Test类

- 右键被测类的java文件，选择“Generate”，打开测试类生成界面。弹出提示，选择Test

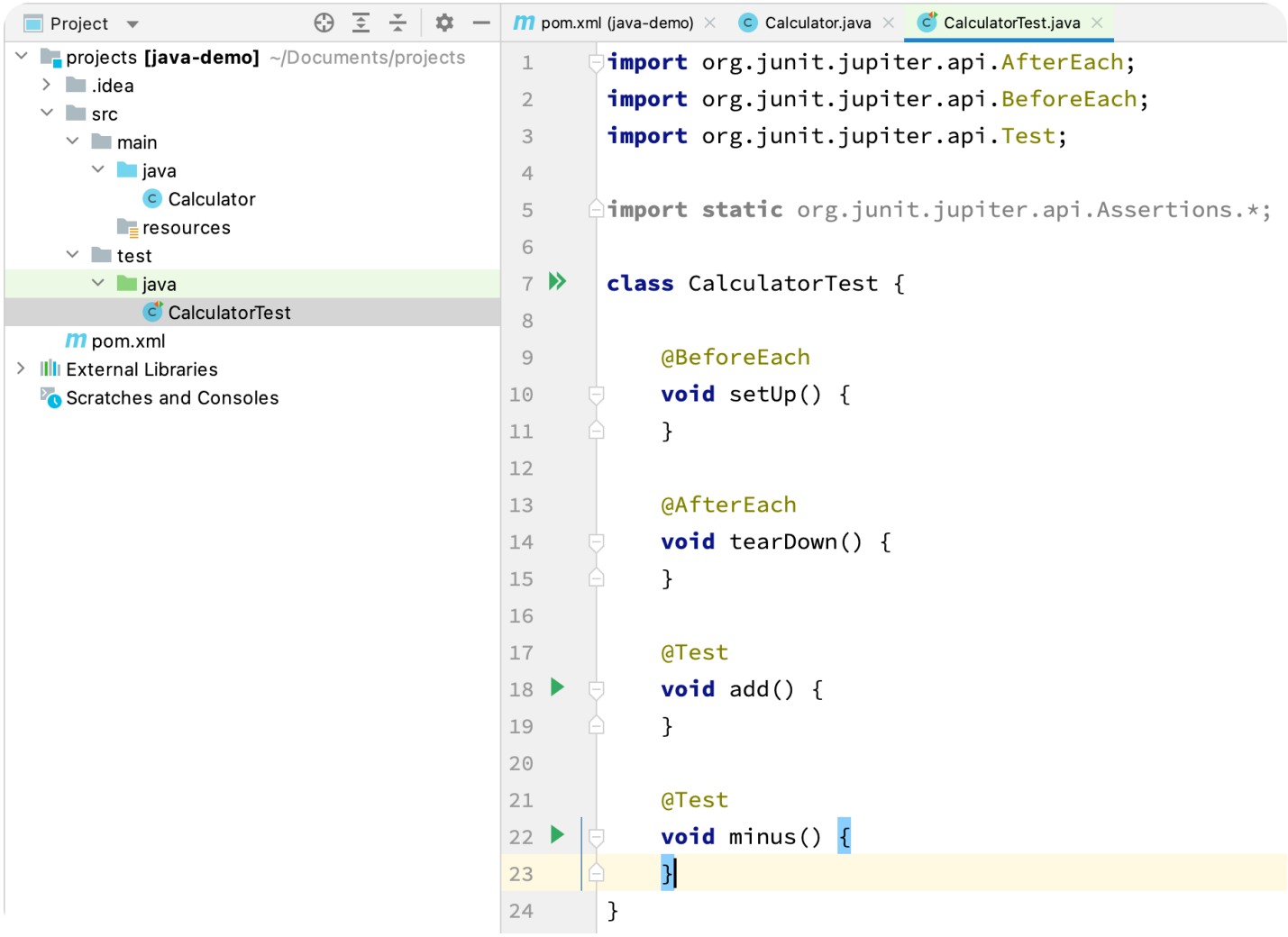




- 选择合适的Junit版本，待测类名以及待测类的成员函数



自动生成对应类的测试代码文件与代码框架



- 若提示junit包导入失败，在pom.xml中加入juint依赖

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    <modelVersion>4.0.0</modelVersion>

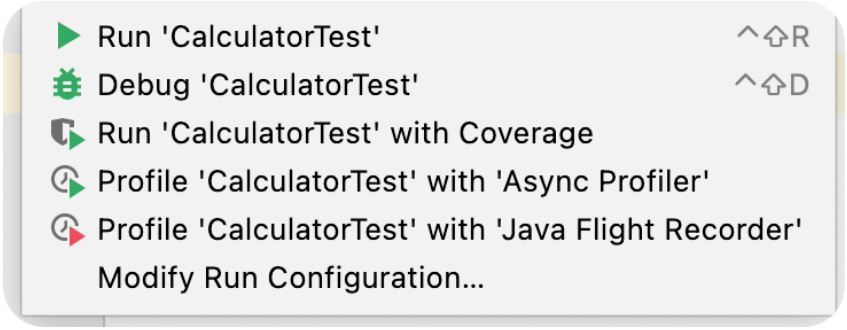
    <groupId>org.example</groupId>
    <artifactId>java-demo</artifactId>
    <version>1.0-SNAPSHOT</version>
    <dependencies>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>RELEASE</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

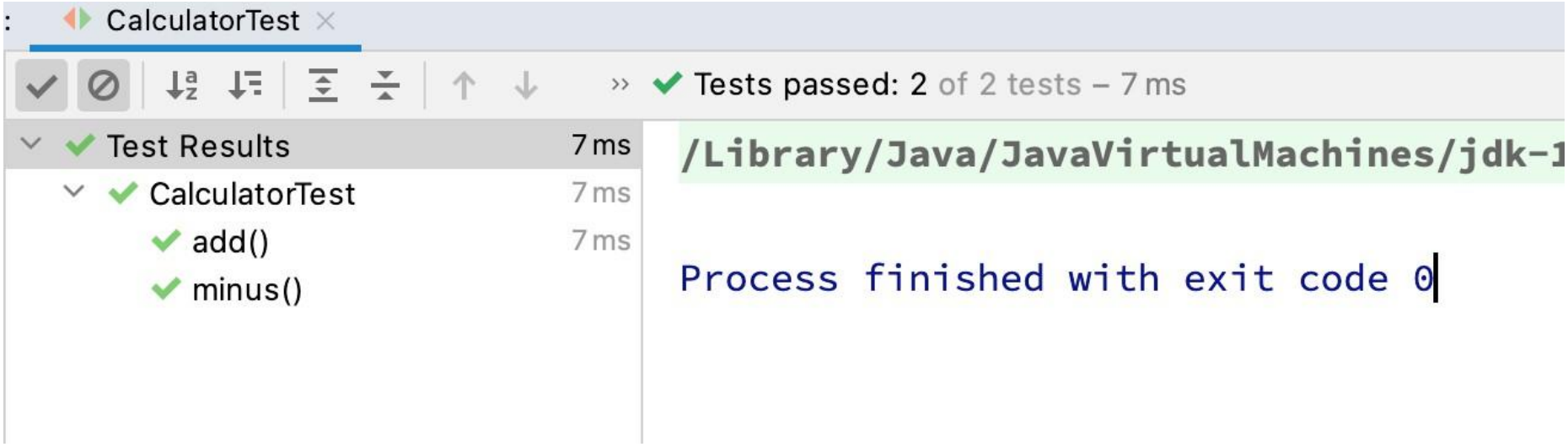
    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
    </properties>

</project>
```

开始测试

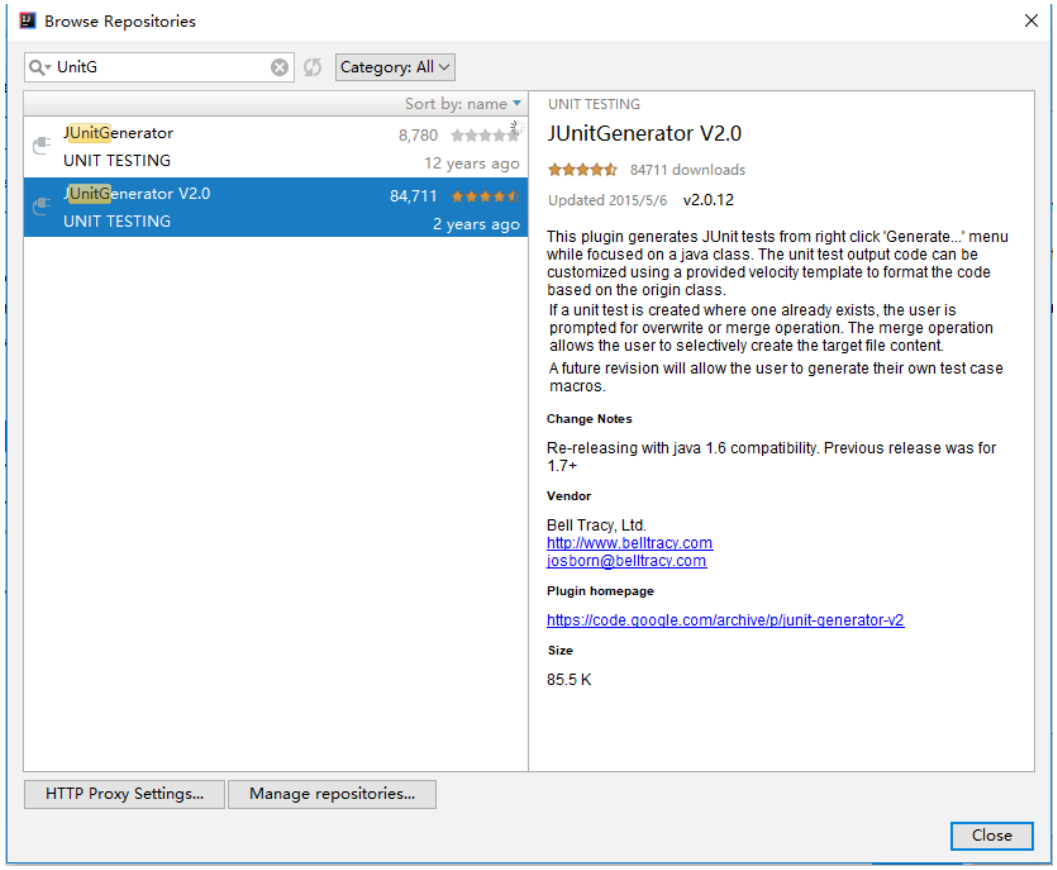
- 右键test类或在test类文件中点击 图标，进行测试





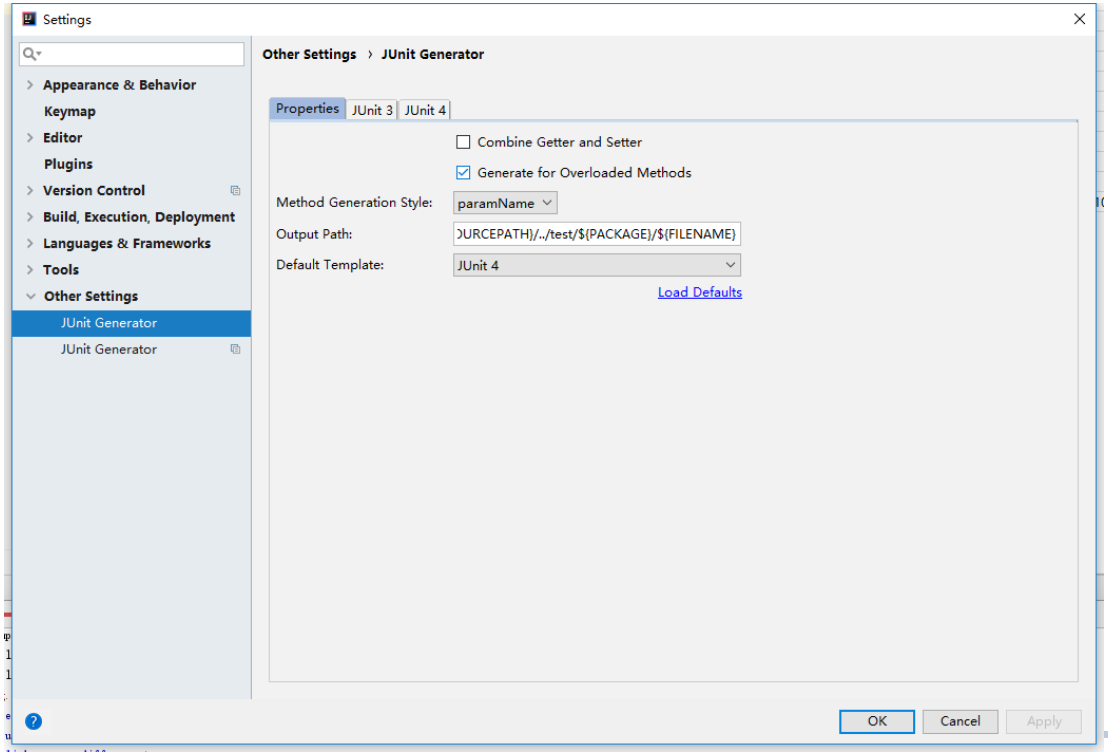
使用插件生成测试代码

- 打开File->Settings->Plugins->Browse Repositories，搜索JunitGenerator并安装（可能要挂代理）。



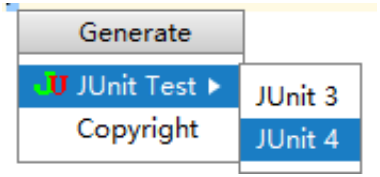
配置插件

- 在File->Settings->Other Settings里找到Junit Generator，修改里面的设置以适应工程：
  - Output Path：自动生成测试代码的输出路径
  - JUnit 4：生成测试文件的模板



生成测试代码

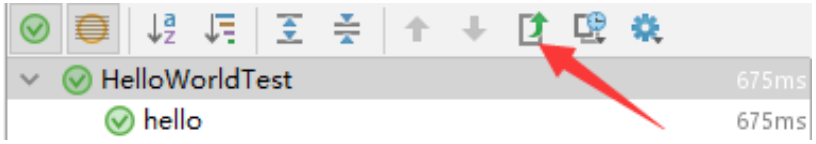
在被测代码文件里右键选择Generate->JUnit Test->JUnit 3/4。或者使用快捷键Alt+Insert。

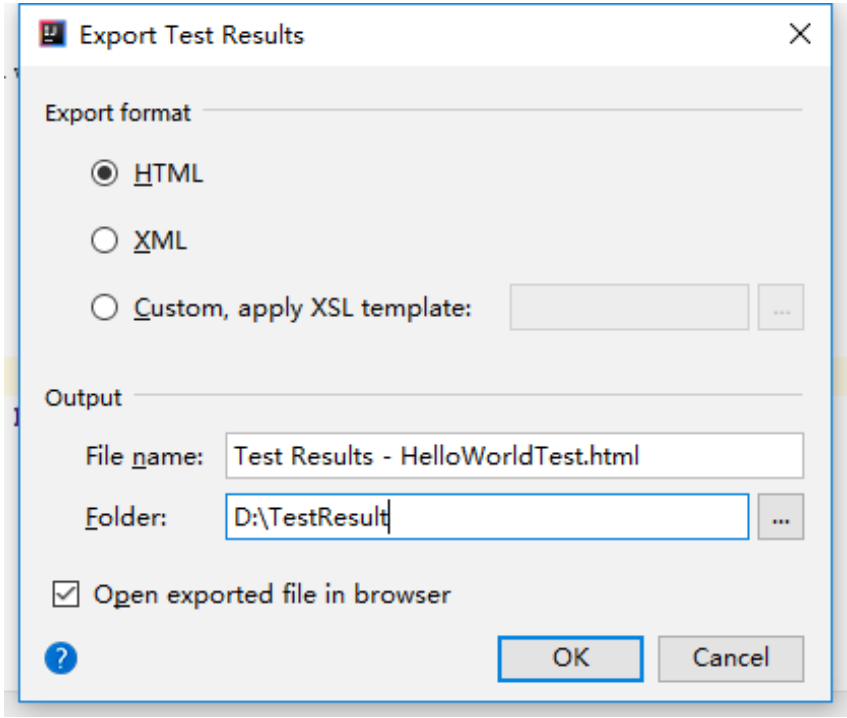


该插件会为测试类下的每个方法生成测试函数，可以使用IntelliJ自带的方式选择只为部分函数生成测试函数

导出测试报告

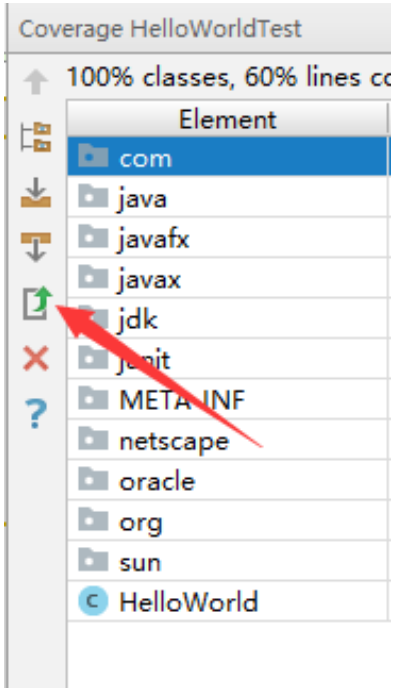
在测试控制窗口界面点击Export Test Results，选择导出格式和路径。





导出测试覆盖率报告

在测试覆盖率窗口点击Generate Coverage Report，选择导出路径。



[all classes] [<empty package name>]

Coverage Summary for Class: HelloWorld (<empty package name>)

Class	Method, %	Line, %
HelloWorld	66.7% (2/ 3)	60% (3/ 5)
HelloWorld$$EnhancerByMockitoWithCGLIB$$78b37baf		
HelloWorld$$EnhancerByMockitoWithCGLIB$$78b37baf$$FastClassByMockitoWithCGLIB$$3553d01e		
HelloWorld$$FastClassByMockitoWithCGLIB$$1a2fa200		
total	66.7% (2/ 3)	60% (3/ 5)

```
1  public class HelloWorld {
2      public int count = 0;
3      public String hello() {
4          return this.getStr();
5      }
6
7      public String getStr() {
8          count++;
9          return "Hello world!";
10     }
11 }
```

generated on 2018-03-09 15:09

其他方法和工具

- 参数化测试：测试用例与测试代码分离，提高测试代码效率，方便测试用例的替换。
  - JUnit4/5
  - TestNG
- Mock（打桩）：模拟对其他模块的调用，避免测试代码被外部代码干扰。(好处：覆盖率、速度、去除影响)
  - Mockito
  - PowerMock
- 变异测试：修改源码使其产生变异体，观察变异体的测试结果，以评判测试用例的充分性。
  - Pitest

参数化测试

- JUnit4中已经包含了对参数化测试的支持，但是代码编写较为繁琐。JUnit5对参数化测试的接口进行了简化。

JUnit4参数化测试代码示例

```
// annotate this test as parameterized test
@RunWith(Parameterized.class)
public class HelloWorldTest {

    // variables to be tested
    private int absInput;
    private int absResult;

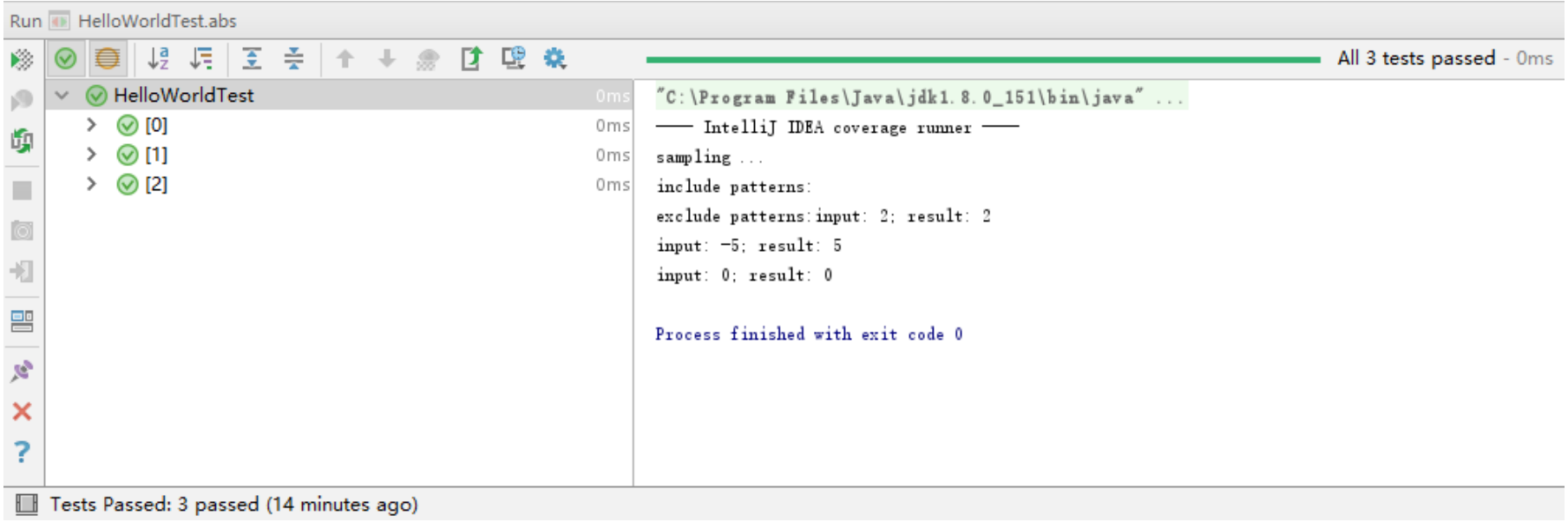
    // parameterized constructor
    public HelloWorldTest(int absInput, int absResult) {
        super();
        this.absInput = absInput;
        this.absResult = absResult;
    }
```

```
// prepare test data
@Parameterized.Parameters
public static Collection<Object[]> data() {
    Object[][] data = new Object[][] {
            { 2, 2 },
            { -5, 5 },
            { 0, 0 }
    };
    return Arrays.asList(data);
}

@org.junit.Test
public void abs() {
    System.out.printf("%dinput: ; result: %d%n", absInput, absResult);
    HelloWorld helloWorld = new HelloWorld();
    int actualResult = helloWorld.abs(absInput);
    assertEquals(absResult, actualResult);
}
```

参数化测试

- @RunWith(Parameterized.class)注解告诉Junit这是一个参数化测试类
- 声明变量作为测试参数
- 实现测试类的构造器，以测试变量为参数
- 实现一个public static类型的函数，返回测试数据（固定写法）
- 测试方法中使用测试参数进行测试



参数化测试

JUnit5中的参数化测试

```
@ParameterizedTest
@ValueSource(ints = { 1, 2, 3 })
void testWithValueSource(int argument) {
    assertTrue(argument > 0 && argument < 4);
}
```

```
@ParameterizedTest
@CsvFileSource(resources = "/two-column.csv", numLinesToSkip = 1)
void testWithCsvFileSource(String first, int second) {
    assertNotNull(first);
    assertNotEquals(0, second);
}
```

two-column.csv

```
Country, reference
Sweden, 1
Poland, 2
"United States of America", 3
```

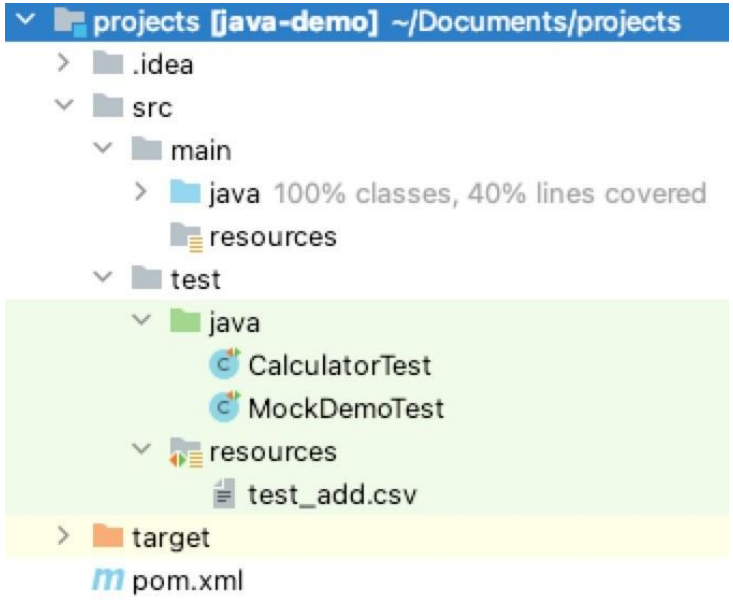
参数化测试

JUnit5中的参数化测试的使用样例

test_add.csv

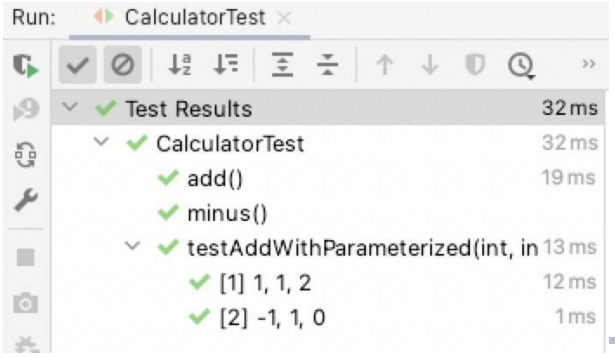
```
num1,num2,expected result
1,1,2
-1,1,0
```

```
public class Calculator {
    public int add(int n1,int n2)
    {
        return n1+n2;
    }
}
```




```
@Test
void add() { assertEquals( expected: 2,calculator.add(1,1)); }

@ParameterizedTest
@CsvFileSource(resources = "/test_add.csv",numLinesToSkip = 1)
void testAddWithParameterized(int num1,int num2,int expected_result)
{
    assertEquals(calculator.add(num1,num2),expected_result);
}
```



1. Java
2. JUNIT
3. EMMA
4. IDEA
5. Mock
6. 智能化测试工具

4.5 Mock工具

- Mockito：Tasty mocking framework for unit tests in Java



https://site.mockito.org/

- 对数据对象的mock的原则：如果不包含业务代码，mock用处不大；如果包含业务代码，则应当用mock将测试用例与其分离。
- 主要接口

- mock()/@Mock: create mock
  - optionally specify how it should behave via Answer/ReturnValues/MockSettings
  - when()/given() to specify how a mock should behave
  - If the provided answers don't fit your needs, write one yourself extending the Answer interface
- spy()/@Spy: partial mocking, real methods are invoked but still can be verified and stubbed
- @InjectMocks: automatically inject mocks/spies fields annotated with @Spy or @Mock
- verify(): to check methods were called with given arguments
  - can use flexible argument matching, for example any expression via the any()
  - or capture what arguments were called using @Captor instead

Mock工具

- **Maven导入：**



Apache Maven

maven.apache.org

```
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <version>3.8.0</version>
</dependency>
```

- **Gradle导入：**

```
repositories { mavenCentral() }
dependencies { testImplementation "org.mockito:mockito-core:3.+" }
```

Mockito代码示例

Calculator.java

```java
public class Calculator {
    public int add(int n1,int n2) { return n1+n2; }
    public int minus(int n1, int n2) { return n1-n2; }
    public String info()
    {
        return this.getInfo();
    }
    public String getInfo()
    {
        return "Test info";
    }
}
```

CalculatorTest.java

```java
private Calculator calculator;

@BeforeEach
void setUp() {
    calculator=spy(Calculator.class);
    when(calculator.getInfo()).thenReturn("Mock info");
}
@Test
void getInfo()
{
    assertEquals( expected: "Mock info",calculator.getInfo());
}
```

变异测试工具

PITest is a state of the art **mutation testing** system, providing **gold standard test coverage** for Java and the jvm. It's fast, scalable and integrates with modern test and build tooling



pox.xml

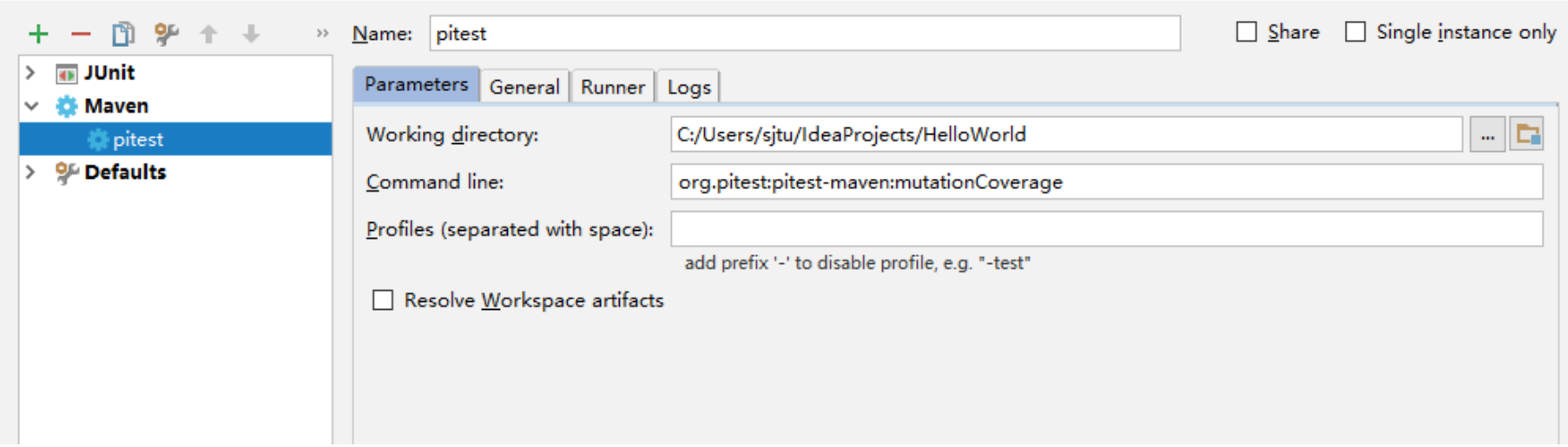
```
<build>
    <plugins>
        <plugin>
            <groupId>org.pitest</groupId>
            <artifactId>pitest-maven</artifactId>
            <version>1.3.2</version>
        </plugin>
    </plugins>
</build>
```

PiTest

- ## Run default mutation coverage goal

```
mvn org.pitest:pitest-maven:mutationCoverage
```

- ## Configurations
  - **Run->Edit Configurations->Add->Maven->Add new Maven build**



- ## Run



PiTest Report

Output / Report in target/pit-reports/YYYYMMDDHHMI

```
- Statistics
>> Generated 6 mutations Killed 1 (17%)
>> Ran 1 tests (0.17 tests per mutation)
- Mutators
> org.pitest.mutationtest.engine.gregor.mutators.ConditionalsBoundaryMutator
>> Generated 1 Killed 0 (0%)
> KILLED 0 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 1
> org.pitest.mutationtest.engine.gregor.mutators.InvertNegsMutator
>> Generated 1 Killed 0 (0%)
> KILLED 0 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 1
> org.pitest.mutationtest.engine.gregor.mutators.ReturnValsMutator
>> Generated 3 Killed 1 (33%)
> KILLED 1 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 2
> org.pitest.mutationtest.engine.gregor.mutators.NegateConditionalsMutator
>> Generated 1 Killed 0 (0%)
> KILLED 0 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 1
```

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage	
1	50%	2/4	17%	1/6

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage	
com.sjtu.yanjiasen4.hello	1	50%	2/4	17%	1/6

Report generated by PIT 1.3.2

Pit Test Coverage Report

Package Summary

com.sjtu.yanjiasen4.hello

Number of Classes	Line Coverage		Mutation Coverage	
1	50%	2/4	17%	1/6

Breakdown by Class

Name	Line Coverage		Mutation Coverage	
HelloWorld.java	50%	2/4	17%	1/6

Report generated by PIT 1.3.2

上海交通大学 SHANGHAI JIAO TONG UNIVERSITY

PiTest Report

Report in target/pit-reports/YYYYMMDDHHMI

HelloWorld.java

```
1   package com.sjtu.yanjiasen4.hello;
2
3   public class HelloWorld {
4       public String hello() {
5   1       return this.getStr();
6       }
7
8       public String getStr() {
9   1       return "Test String";
10      }
11
12      public int abs(int n) {
13  4       return n >= 0 ? n : (-n);
14      }
15  }
```

Mutations

5 1. mutated return of Object value for com/sjtu/yanjiasen4/hello/HelloWorld::hello to (if (x != null) null else throw new RuntimeException) → KILLED

9 1. mutated return of Object value for com/sjtu/yanjiasen4/hello/HelloWorld::getStr to (if (x != null) null else throw new RuntimeException) → NO_COVERAGE

13
1. changed conditional boundary → NO_COVERAGE
2. removed negation → NO_COVERAGE
3. negated conditional → NO_COVERAGE
4. replaced return of integer sized value with (x == 0 ? 1 : 0) → NO_COVERAGE

Active mutators

- INCREMENTS_MUTATOR
- VOID_METHOD_CALL_MUTATOR
- RETURN_VALS_MUTATOR
- MATH_MUTATOR
- NEGATE_CONDITIONALS_MUTATOR
- INVERT_NEGS_MUTATOR
- CONDITIONALS_BOUNDARY_MUTATOR

Tests examined

- com.sjtu.yanjiasen4.hello.HelloWorldTest.hello(com.sjtu.yanjiasen4.hello.HelloWorldTest) (140 ms)

Report generated by PIT 1.3.2

Mutate Operator

- Conditionals Boundary Mutator
  - < / <=, > / >=
- Negate Conditionals Mutator
  - == / !=, <= / >, > / <=
- Remove Conditionals Mutator
  - a == b / true
- Math Mutator
- Increments Mutator
- Return Values Mutator

1. Java
2. JUNIT
3. EMMA
4. IDEA
5. Mock
6. 智能化测试工具

如何利用大模型应用来生成测试用例

1.语义化代码理解

2.多维度用例生成

路径覆盖：自动识别代码分支条件

等价类划分：生成典型/边界值组合

突变测试：植入常见代码缺陷模式

大模型的软件测试有效性评估

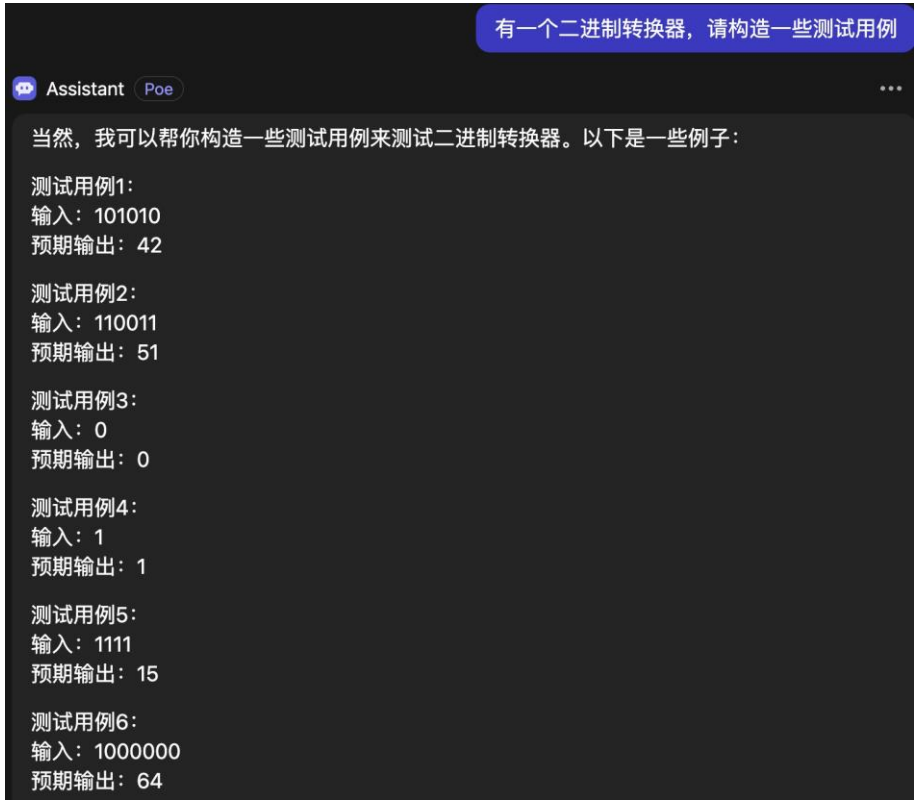
识别冗余测试用例

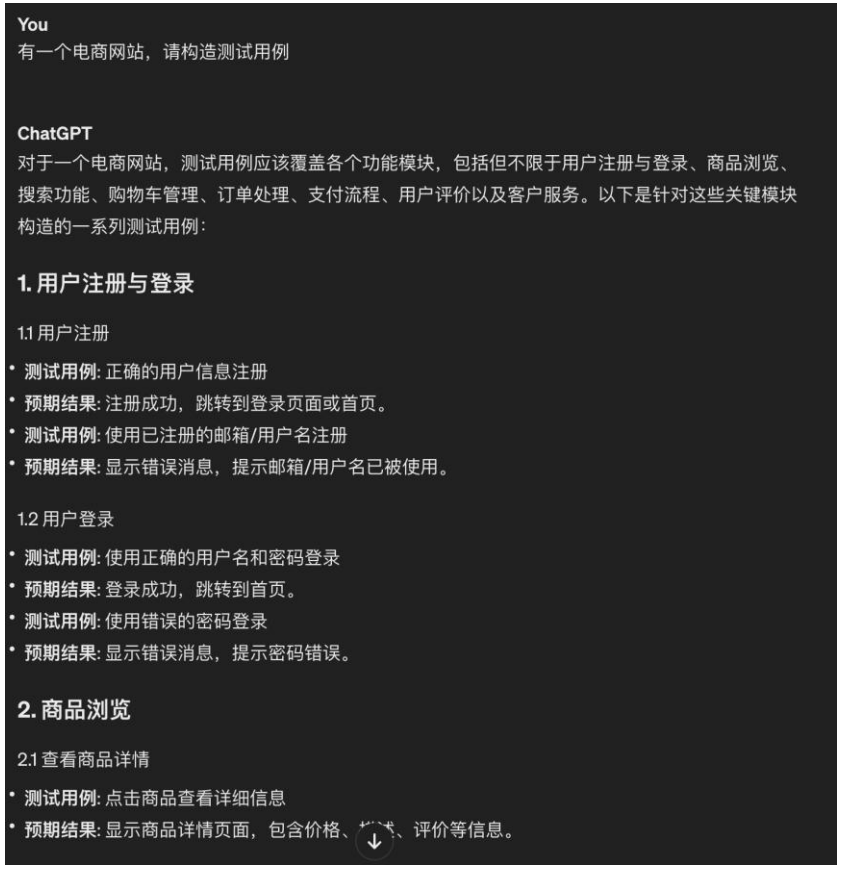
检测断言缺失场景

发现隐藏的异常路径

大模型测试示例

随着大语言模型技术的发展，大量的技术人员选择使用其完成测试用例准备（包括黑盒测试、单元测试用例生成、测试预言生成、系统级测试输入生成）、测试报告分析、程序调试和修复等任务。





上海交通大学 SHANGHAI JIAO TONG UNIVERSITY

大模型测试示例

正则表达式黑盒测试用例生成：

我现在有一个正则表达式生成器，请基于等价类的方式，生成对应的测试用例

ChatGPT

为了生成正则表达式生成器的测试用例，我们可以基于等价类划分的方法来进行。等价类划分是一种软件测试设计技术，它将输入数据的集合划分为有效和无效的等价类。对于每个等价类，只需选择一个代表性的值进行测试即可。对于正则表达式生成器，输入数据主要是用户输入的模式（pattern）和待匹配的字符串（string），输出则是匹配结果的布尔值或具体的匹配项。

以下是根据等价类划分法设计的测试用例：

1. 测试有效等价类

1.1 简单字符串匹配

- **输入:** 模式 "hello", 字符串 "hello"
- **期望输出:** 成功匹配

1.2 特殊字符匹配

- **输入:** 模式 "\d+" (匹配一个或多个数字), 字符串 "12345"
- **期望输出:** 成功匹配

1.3 分组和选择

- **输入:** 模式 "(hello|world)", 字符串 "hello"
- **期望输出:** 成功匹配

1.4 贪婪与非贪婪匹配

- **输入:** 模式 "a+" (贪婪), 字符串 "aaa"
- **期望输出:** 成功匹配所有 "a"
- **输入:** 模式 "a+?" (非贪婪), 字符串 "aaa"
- **期望输出:** 成功匹配单个 "a"

2. 测试无效等价类

2.1 不匹配的模式和字符串

- **输入:** 模式 "hello", 字符串 "world"
- **期望输出:** 匹配失败

2.2 错误的正则表达式

- **输入:** 模式 "\z" (不存在的转义序列), 字符串 "abc"
- **期望输出:** 抛出正则表达式错误

2.3 空字符串和空模式

- **输入:** 模式 "", 字符串 "hello"
- **期望输出:** 根据正则表达式生成器的设计，可能是匹配失败或特定行为
- **输入:** 模式 "hello", 字符串 ""
- **期望输出:** 匹配失败

3. 边界条件测试

3.1 模式和字符串在边界条件下的匹配

- **输入:** 非常长的模式和字符串
- **期望输出:** 根据性能和处理能力，可能是成功匹配或处理超时

3.2 特殊情况的字符处理

- **输入:** 包含换行符、制表符等特殊字符的模式和字符串
- **期望输出:** 成功匹配，如果正则表达式生成器支持这些特殊字符

通过这些测试用例，可以系统地验证正则表达式生成器在不同情况下的表现，确保其正确性和鲁棒性。

作业一：黑盒测试

目的：熟练掌握黑盒测试方法原理、测试过程、测试环境与工具。

要求：

1. **选择一个大模型聊天应用（或工具），如DeepSeek，腾讯元宝，豆包等。要求设计至少五个文本生成任务，如计算器，word count，编程等，要求其中至少三个任务可以使用边界值、等价类、决策表中的方法生成黑盒测试用例。参考被测软件《需求规格说明书》撰写要求，简单撰写一份《大模型聊天应用任务说明书》。**
2. **对你设计的5个文本生成任务进行黑盒测试，其中至少三个任务需要使用边界值、等价类、决策表三种测试方法中任意两种进行分析与测试，要求形成《测试计划》和《测试报告》。**
3. **（可选）可以先使用大语言模型（如ChatGPT）生成测试用例，在用其他方法对生成的测试用例进行分析或补充（需在测试计划与测试报告中体现此过程）。**
4. **执行测试用例并记录测试结果，比较不同生成方式的区别，并完成相应的文档：**
    1. **《任务说明书》（参考被测软件《需求规格说明书》）**
    2. **《测试计划》**
    3. **《测试报告》**

备注：

1. **在测试报告中包含小组成员贡献与比例**
2. **提交方式：将项目打包成zip，以小组编号命名后提交到canvas中**

大模型测试示例

利用大模型应用来生成测试代码

- 当前部分大模型已经可以直接生成测试代码，填充生成的测试用例输入。
- 利用大模型对测试代码优化
  - 参数化测试重构
  - 测试依赖管理
  - Mock对象智能生成

```python
# 模型生成的单元测试代码示例
import unittest

class TestDiscount(unittest.TestCase):
    def test_vip_user(self):
        self.assertAlmostEqual(calculate_discount(500, "vip"), 400)

    def test_high_price(self):
        self.assertEqual(calculate_discount(1500, "regular"), 1350)

    def test_invalid_input(self):
        with self.assertRaises(TypeError):
            calculate_discount("invalid", 123)
```



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

作业二：单元测试

选取一个包含源码的软件以白盒测试的方式，设计测试用例并进行单元测试。

要求：

1. 针对你选的软件源码，使用DD路径与数据流分析方法生成单元测试用例，撰写《测试计划》。要求用例执行的代码覆盖率不低于95%，并分析两种方法的覆盖率。
2. 执行测试用例，并根据结果形成《测试报告》。
3. 使用大语言模型应用，针对选择的软件源码，自动化生成单元测试用例，并从覆盖率和测试用例复杂度等角度对其进行评估，形成《大模型单元测试用例自动生成分析报告》。
4. （可选）如你认为大模型自动生成的单元测试用例更优化，可根据该测试用例改进你的《测试计划》，并在文档中体现。
5. 提交《测试计划》、《测试报告》、《大模型单元测试用例自动生成分析报告》以及测试代码和被测程序源码。

备注：

推荐使用课程项目或成熟稳定维护的开源项目等进行测试，要求源码代码行数不低于400行

如有需要，可以使用Mock工具体现打桩

推荐使用参数化测试

报告中只需提供关键代码的DD路径与数据流分析

在测试报告中包含小组成员贡献与比例

如何验证大模型生成的测试正确性?

多模型协同验证技术

- 同时使用GPT、LLAMA、Deepseek
- 投票机制

差分测试（Differential Testing）

- 对不同的测试用例生成进行多版本的对比

变异测试增强

- 植入更多的变异

基于路径覆盖代码覆盖来检查

- 结合人工检查

大模型测试生成测试代码案例

VIP用户有20%折扣，普通用户如果价格超过1000则有10%折扣，否则没有折扣。

```python
import unittest
import ast
from io import StringIO
import sys

# 被测函数
def calculate_discount(price: float, user_type: str) -> float:
    """计算商品折扣
    Args:
        price: 商品价格
        user_type: 用户类型(vip/regular)
    """
    if user_type == "vip":
        return price * 0.8
    elif price > 1000:
        return price * 0.9
    return price
```

```python
# 大模型生成的测试代码
generated_test_code = """
import unittest

class TestDiscount(unittest.TestCase):
    def test_vip_discount(self):
        self.assertAlmostEqual(calculate_discount(500.0, 'vip'), 400.0)

    def test_regular_high_price(self):
        self.assertEqual(calculate_discount(1500.0, 'regular'), 1350.0)

    def test_invalid_price_type(self):
        with self.assertRaises(TypeError):
            calculate_discount('invalid', 'vip')

    def test_edge_case_1000(self):
        self.assertEqual(calculate_discount(1000.0, 'regular'), 1000.0)
"""
```

下面演示一个大模型生成软件测试代码的完整案例，包含动态验证和变异测试：

正在思考...

> “以下是一个利用大模型生成软件测试代码的完整案例，包含静态验证、动态执行、覆盖率分析和变异测试：

一些可能的建议与思考

结合**形式化验证、动态执行监控和人机协同机制**

实际应用中建议：

- 对核心模块采用多重校验机制
- 建立测试用例版本追溯系统
- 实施分级信任机制（关键/非关键测试区分处理）
- 定期进行对抗性测试挑战

智能辅助+人工监督

上海交通大学 SHANGHAI JIAO TONG UNIVERSITY

补充思考：如何测试大模型应用













补充思考：如何测试大模型应用

一、系统与基础设施测试

- 分布式训练框架测试
- GPU显存泄露检测
- 服务弹性伸缩测试

二、算法模型层——智能化AI功能相关的测试

1. 模型基础能力评估

- 模型准确性

2. 动态行为测试

- 思维链（CoT）合理性验证：
- 上下文窗口测试
- 多模态对齐测试

三、 AI应用的性能基准测试

- 动态批处理效率测试
- 冷启动预热测试，平均响应时延测试

上海交通大学 SHANGHAI JIAO TONG UNIVERSITY

报告示例

某支撑软件与可视化软件
测试计划

目录

1、简介
1. 编写目的
2. 项目背景
3. 测试范围
4. 参考文档
2、测试参考文档和测试提交文档
1. 测试参考文档
2. 测试提交文档
3、测试进度
4、测试资源
1. 人力资源
2. 测试环境
3. 测试工具
5、测试策略

1. 功能测试
   1. XXX系统管理软件功能测试
   2. XXX系统实时通信软件功能测试
   3. 目标三维动态显示模型及可视化软件功能测试
   4. XXX三维动态显示模型及可视化软件功能测试
   5. XXX交会显示软件功能测试
   6. XXX系统数据管理软件功能测试
   7. XXX文件与AutoCAD、Creator的转换软件功能测试
2. 性能测试
   1. 通讯性能测试
   2. 实验性能测试
   3. 数据库性能测试
   4. 显示性能测试
   5. 其它性能测试
3. 故障测试
   1. 网络故障
   2. 服务器故障
   3. 管理计算机故障
   4. 计算节点机故障
   5. 图形工作站故障测试
4. 安全性测试
   1. 管理软件安全性测试
   2. 数据管理软件安全性测试

上海交通大学 SHANGHAI JIAO TONG UNIVERSITY

1、简介

- 1. 编写目的
- 为了全面、系统地对“XXX支撑软件与可视化软件”进行评估与测试，从而保证系统长期稳定的运行，组织对该软件进行系统的总体综合测试。
- 2. 项目背景
- “XXX验证平台”是中国兵器工业XXX研究所承担的国防基础科研项目。XXX所与XXX大学合作，对该项目的关键技术——XXX支撑技术和可视化技术展开研究。本项目就是在此背景下开展的，通过应用虚拟试验技术、计算机网络技术、可视化技术、数据库技术等，研制“XXX的支撑软件和可视化软件”，并最终集成在“XXX中。
- 本软件系统由以下7个二级软件组成：

  系统管理、实时通讯、三维显示等

1.3测试范围

本次系统测试将采用开发相关测试软件和利用成熟的测试工具相结合的方式，现场对系统进行测试。按照系统需求任务书的要求，分别进行系统的功能测试、性能测试、故障测试以及安全性测试等相关内容。

本次测试的主要目标包括：

系统软件单元功能测试——根据软件详细设计文档的要求，完成各个软件模块的单元功能测试工作；

系统软件集成功能测试——根据软件需求规格说明的要求，完成软件功能测试工作；

系统软件性能测试——根据技术协议书和软件需求规格说明的要求，完成软件性能测试工作；

系统故障测试——对软件系统在网络故障、数据库故障等异常情况下进行测试工作；

安全性测试——对系统用户登陆、密码保护等安全管理进行测试工作。

- 1.4参考文档
- 《GB 8566-88 计算机软件开发规范》
- 《GJB 438A-97 武器系统软件开发文档》

2、测试参考文档和测试提交文档

- 2.1测试参考文档
- 本测试计划参照GB8567——88标准编制，下表列出了制定测试计划时所使用的文档，并标明了各文档的可用性：

2.2测试提交文档

在测试完成后，需要提交给用户相关的测试评估报告与改进报告，并提交详细的测试报告。

3、测试进度

测试活动	计划开始日期	实际开始日期	结束日期
单元功能测试	2006年 3月16日	2006年3月16日	2004年3月26日
集成功能测试	2006年3月27日	2006年3月27日	2006年3月31日
性能测试	2006年4月2日	2006年4月3日	2006年4月5日
故障测试	2006年4月8日	2006年4月8日	2006年4月9日
安全测试	2006年4月10日	2006年4月10日	2006年4月11日
对测试进行评估	2006年4月15日	2006年4月16日	2006年4月18日
用户验收测试	2006年5月11日		

4.1 测试资源

4.1 人力资源

姓名	角色	具体职责或注释
胡　飞	总体策划人、方案设计人	策划总体规模，测试内容，规划测试方案
A	方案设计人、测试技术设计人	制定测试方案，确定测试深度，测试技术设计
B	计划人、记录人	计划测试进程，记录测试情况
C	计划人、测试人、记录人	计划整个进程以及各个阶段的进度安排、重点任务；测试并记录测试情况
D	计划人、测试人、记录人	测试并记录测试情况
E	测试人、记录人	测试并记录测试情况
F	测试人、记录人	测试并记录测试情况
G	测试人、记录人	测试并记录测试情况

4.2 测试环境

下表列出了测试的系统环境：

机器名 环境	数据库服务器	图形工作站	管理计算机
软件环境（相关软件操作系统等）	Windows XP、Oracle 9i		Windows XP
硬件环境（设备、网络等）	Xeon Mp 2.7×2 CPU/4smp/2GB/DVD/1000M×2	HP Workstation xw6200	HP Workstation xw6200

4.3 测试工具

下表列出此项目测试中使用的工具：

用途	工具名称	生产厂商/自产	版本
压力与性能测试	Jmeter	Jakarta	1.9.1
压力测试网络检测	Webstress	国外软件	6.18
申请一定量的内存	Men	自产	1.0

五、测试策略

5.1 功能测试

由于对测试对象的功能测试应侧重于所有可直接追踪到用例或业务功能和业务规则的测试需求。这种测试的目标是核实数据的接受、计算、处理和输出是否正确，以及业务规则的实施是否恰当，所以此次的功能测试全部为黑盒测试，测试的主要目的是检测以下错误：

1 是否有不正确或遗漏的功能？

2 在接口上，输入是否能正确的接受？能否输出正确的结果？

3 是否有数据错误或外部信息（例如数据文件）访问错误？

4 是否有初始化或终止性错误？